

10-1 Records (structs)

- ✚ C++ structures allow you to define custom data types that group related variables of the same or different data types under a single name, making complex data easier to organize and manage.
- ✚ C++ provides a structured data type called **struct** to group items of different types.
- ✚ Grouping components that are related but of different types offers several advantages. For example, a single variable can pass all the components as parameters to a function.
- ✚ **struct**: A collection of a fixed number of components in which the components are accessed by name. The components may be of different types. The components of a **struct** are called the members of the **struct**.

The general syntax of a **struct** in C++ is:

```
struct structName
{
    dataType1 identifier 1;
    dataType2 identifier 2;
    .
    .
    .
    dataType n identifier n;
};
```

A semicolon at the end of the **struct** is, therefore, a part of the syntax.

Example define the **struct** :

```
struct houseType
{
    string style;
    int numofBedrooms;
    int numofCarsGarage;
    int yearBuilt;
    double price;
    double tax;
};
```

Members

تعريف نوع بياني سجل يتضمن 6
members

houseType tempHouse; ← tempHouse اعلان عن متغير اسمه
من نوع houseType

**** A variable can also be declared with the definition of the structure ****

10-2 Accessing struct Members

✚ To access a structure member (component), you use the **struct** variable name together with the member name; these names are separated by a dot (period).

✚ The syntax for accessing a **struct** member is:

structVariableName.memberName

In C++, the dot (.) is an operator called the **member access operator**.

✚ Structure members are accessed using **dot operator (.)** and a new value can also be assigned using **assignment operator**.

Example(1) :

Write program to save student information in record that involve the name and degree of this student.

```
#include <iostream>
using namespace std;
struct student{
    string name;
    int degree;
};

int main(){
    student stu_1 = {"Ali", 80};

    // Accessing members
    cout << stu_1.name << " ";
    cout << stu_1.degree << endl;

    // Updating members
    stu_1.degree = 99;

    // Accessing members again
    cout << stu_1.name << " ";
    cout << stu_1.degree ;
```

```
    return 0;
}
```

10-3 Member Functions

✚ In C++, we can declare the function inside the structure. They are called member functions while the variables are called data members. As shown in the following example:

```
#include <iostream>
using namespace std;
struct student{
    string name;
    float degree1;
    float degree2;
    // Member function
    float average(){
        return (degree1 + degree2)/2;
    }
};

int main(){
    student stu_1 = {"Ali", 81, 90};

    // Accessing members
    cout << "the name is : "<< stu_1.name << "\n";

    // Call member function using (.) operator
    cout << "the average is : "<< stu_1.average() << endl;
    return 0;
}
```

Output:

the name is : Ali

the average is : 85.5

Example: write program in C++ to create a record to store student information and use member functions to:

- 1- Input student data.
- 2- Display student data
- 3- Calculate the average mark.

Sol.

```
#include <iostream>
using namespace std;
```

```
struct Student
{
    string name;
    int id;
    float mark1, mark2, mark3;

    // Member function to input data
    void input()
    {
        cout << "Enter student name: ";
        cin >> name;
        cout << "Enter student ID: ";
        cin >> id;
        cout << "Enter three marks: ";
        cin >> mark1 >> mark2 >> mark3;
    }

    // Member function to calculate average
    float average()
    {
        return (mark1 + mark2 + mark3) / 3;
    }

    // Member function to display data
    void display()
    {
        cout << "Student Name: " << name << endl;
        cout << "Student ID: " << id << endl;
        cout << "Average Mark: " << average() << endl;
    }
};

int main()
{
    Student s;
    s.input();
    s.display();

    return 0;
}
```

Output:

Enter student name: Ali
Enter student ID: 9910099
Enter three marks: 90 100 70

Student Name: ali
Student ID: 9910099
Average Mark: 86.6667

لاحظ استدعاء دالة داخل دالة اخرى

10-4 Pointers in C++

Pointers in C++ are a symbolic representation of addresses.

- Pointers store the address of variables or a memory location.
- Can be reassigned to point to another variable
- Can be nullptr
- Requires dereferencing to access the value

Syntax

```
datatype *var_name;
```

Example:

This program declares an integer variable 'x' initialized with value 10 and a pointer variable named 'myptr'. The memory address of x is assigned to myptr. Then it prints the value of x, the address stored in myptr, and the value of x obtained by dereferencing the pointer myptr.

```
#include <iostream>
using namespace std;
int main(){
    int x = 10; // variable declared
    int* myptr; // pointer variable

    // storing address of x in pointer myptr
    myptr = &x;

    cout << "Value of x is: ";
    cout << x << endl;

    // print the address stored in myptr pointer variable
    cout << "Address stored in myptr is: ";
    cout << myptr << endl;

    // printing value of x using pointer myptr
    cout << "Value of x using *myptr is: ";
    cout << *myptr << endl;

    return 0;
}
```

Output:

Value of x is: 10

Address stored in myptr is: 0x6ffe44

Value of x using *myptr is: 10